RL Replay Analyser: Aggregated Behaviour Analytics for Rocket League Players

Abstract

Rocket League players often evaluate performance from individual matches; however, a player's behaviour can vary heavily match-to-match, which can hide the persistent tendencies a player may have that define their long-term strengths and weaknesses. This project is a web-based analytics system that aggregates replay-derived statistics across many of a user's games and visualises these tendencies through comparison against teammates/opponents and rank averages, while additionally applying a Random Forest classifier, trained on manually labelled data, to assign them an interpretable playstyle category.

Motivation

Rocket League is a fast, high-variance competitive game where individual match outcomes are influenced by many short-term factors (teammate synergy, opponent style, one-off mistakes, kick-off variance, confidence). As a result, conclusions drawn from individually reviewing one or two replays is an unstable method of diagnosing a player's tendencies and flaws. The same player may play a highly defensive game in one match, yet an over-aggressive one in another, despite having a consistently average behaviour across a larger sample.

Common publicly available tools exist and are excellent for single-game analytics but are weaker at providing the user ways to profile their playstyle. This creates a gap between highly detailed per-match breakdowns, and the broader, persistent behavioural trends players actually need to improve efficiently. For many players, coaching is the most reliable route to receive this behavioural feedback, however it is not affordable or necessary for most casual-to-serious players. Therefore, I aim to provide an accessible middle ground: automatically aggregated feedback that is derived from statistics and easy to interpret, guiding users into forming their own useful conclusions.

Requirements Capture

Interviews

To better understand how players interact with existing analytics tools and how they would characterise different playstyles, I conducted four interviews with Rocket League players of varying skill levels. Full details are contained in Appendix A, with Interview 3 being the most detailed and impactful. The key takeaways were as follows:

- **Data Overview.** All interviewees mentioned using "Ballchasing.com" as a method of improving, which provides statistics for individual replays in the form of charts. This reinforces the idea that such a section would be useful, aggregating multiple replays rather than individual breakdowns.
- **Replay Volume.** An interviewee mentioned that upload limits on ballchasing.com led to less frequent use. By including a built-in parser and not permanently storing replay data server-side, an indefinite number of replays can theoretically be supported, providing larger samples and making behavioural trends easier to identify reliably.
- **Comparison to Rank Averages.** An interviewee suggested comparison to average players at certain ranks, additionally suggesting spider diagrams to represent this, allowing users to easily spot outliers in their gameplay that may indicate flaws.

- **Playstyle Classification.** Several of the same playstyles were commonly mentioned across interviews with similar characteristics representing each, confirming that a supervised classification approach with manually labelled data would be viable and appropriate.

Previous Solutions

Alongside the interviews, existing tools were analysed to further inform the requirements.

Ballchasing.com. The main inspiration for this project; a public replay database where players can upload and analyse their replays. Its detailed statistical visualisation will inform my Data Overview section. Notable features include **category-based filtering** to prevent the user being overwhelmed, and abundant use of **bar charts** with clear legends and contrasting colours to provide intuitive player comparisons.

Rocket League Tracker (rocketleague.tracker.network). Primarily a rank-tracking site, though it includes useful features such as **stacked bar charts** to represent **data ratios**, which I may use in my Data Overview section. It attempts to visualise playstyle through the ratio of goals, saves, and assists, which I consider insufficient and unindicative of how a player truly plays.

Hollow Knight Save Completion Analyzer. Though unrelated to Rocket League, this site parses save files in a similar way to how my system will parse replays. Useful inspiration includes presenting the **folder path** of uploaded files for easy copying and displaying **basic header details** so the user can confirm correct files were uploaded.

Additional details and evaluations can be found in Appendix B.

List of Requirements

The most significant, overarching requirements are outlined below. For the full comprehensive list, refer to Appendix C.

Functional Requirements

Uploading Replays

- Allow the user to upload multiple Rocket League replays through file upload
- Allow users to enter public game ID/URL from ballchasing.com as an alternative
- Only allow valid replay files to be uploaded, presenting appropriate error messages otherwise
- Parse valid replays in the backend with an appropriate library
- Display “header” information for each uploaded replay so user can confirm they are correct
- Ask the user for basic information to help inform future sections of the system

Data Overview

- Visualise average of useful gameplay statistics using appropriate charts
- Use the average values per teammate and per opponent for informative comparison
- Allow filtering between categories of data

Playstyle Classification

- Use supervised model to classify the user’s playstyle
- Manually label training data
- Provide model confidence to show similarity to other playstyles, allowing user to grasp their playstyle as a mixture of all of them, rather than a single label

- Present feature importance metrics to give user insight into which aspects of their gameplay drove the classification
- Give general advice using suggestions from interviews

Rank Comparison

- Compare the user's averaged statistics with average values of their rank and any other rank
- Highlight user's statistics that are outliers in comparison to their rank's average values
- Visualise comparisons using spider charts
- Source data from large, pre-computed dataset to avoid repeated API calls

Non-Functional Requirements

User Interface

- Intuitive, responsive, and clear interface, with consistent and contrasting colours, adequate spacing, and clearly labelled charts
- Outliers in the Rank Comparison section highlighted with contrasting colours
- Progress visualisation, such as a loading bar, for longer tasks

Performance

- Support large batches of replays with an average loading time of under 2–3 seconds per replay
- Use pre-computed dataset for quicker rank comparison loading
- Cache parsed data for quicker repeated computation

Accuracy

- Use carefully selected features to train the model for high accuracy and lower noise
- Test models using standard methods to ensure sufficient accuracy (75%+ given the subjective nature of playstyle classification)
- Use recent data to ensure comparisons and models reflect today's standards across ranks

Reliability and Security

- Error handling keeps the site responsive, if possible, with suitable error messages otherwise
- User's data is only processed, never stored server-side

The following section will describe the design decisions made to satisfy these requirements.

Design

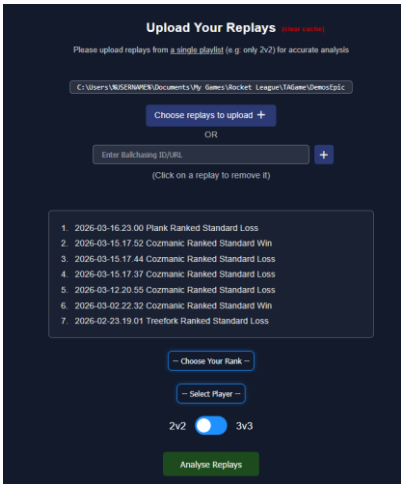
Project Methodology Overview

Before implementation began, the list of requirements was formed and each section of the project was designed against them, starting with the user interface. Beginning with the UI allowed each feature to be visualised as it was designed and implemented, keeping the whole picture in view throughout. Design documents were referred back to during implementation to keep the process structured. Implementation focused on building the main features first as proofs of concept before refining them, preventing the risk of over-investing in a single feature at the expense of missing critical parts of the project. Features were tested immediately upon completion, with the system as a whole verified after each significant change to ensure continued expected behaviour. The following sections detail the design decisions made across the UI, frameworks and libraries, and each feature of the system.

Frameworks and Libraries

React was chosen for the frontend due to its component-based architecture, which naturally suits the modular section layout of the site, and its state management making all user interactions straightforward to implement. **Recharts** integrates cleanly with React and supports all chart types required, including spider charts. For the backend, **FastAPI** was chosen as it is the fastest Python backend framework, with asynchronous support allowing multiple replays to be parsed simultaneously, which is critical for meeting performance requirements. **Python** was the natural backend choice regardless, as it is the most suitable language for data manipulation and machine learning. **Carball**, an open-source Python package specifically built for parsing Rocket League replays, was the only viable option for replay parsing. Finally, **Scikit-learn** was used for the playstyle classifier, as it excels at supervised classification with numerical data and natively supports feature importance and model confidence extraction.

Replay Upload Page



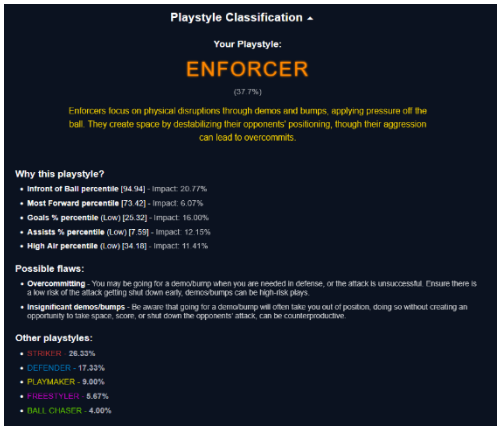
The upload page is intentionally minimal on first load, presenting only upload instructions and inputs to give the user a single task at a time. Two upload methods are available: a file upload, where the user submits their own .replay files directly, validated and parsed using the Carball parser; and a Ballchasing.com URL or ID upload, where the system retrieves replay details automatically via the API. All uploaded replays display their name, game mode, and date, with the map name used as a fallback where no replay name exists, so the user can confirm correct uploads. Replays can be removed by clicking them in the list. Before submitting, the user selects the player to be analysed, their rank, and the game mode from dropdowns. A loading bar displays parsing progress during submission.

Data Overview

The Data Overview section presents statistics across five or six categories: Core (shots, goals, assists, and saves), Boost, Movement, Positioning, Demos, and optionally Possession, which is only available for file uploads as Ballchasing.com does not store per-player possession data. The full list of statistics per category is in Appendix D.3. Most statistics are represented as bar charts showing the user, teammate average, and opponent average, with each group coloured consistently across all charts for immediate identification. Proportional statistics such as time spent at different heights are represented as stacked bar charts, while statistics relative to teammates are represented as pie charts. Charts are clearly titled, and the active category is highlighted with a blue glow.

Playstyle Classification

The goal of this section is to assign the user a playstyle label based on their averaged statistics to highlight potential patterns and flaws. Since playstyle classification is inherently subjective, justification is provided through statistics so the user can learn from the data regardless of whether the label aligns with their own perception. Each playstyle is assigned a distinct bright colour carried through the interface to give it a unique identity.

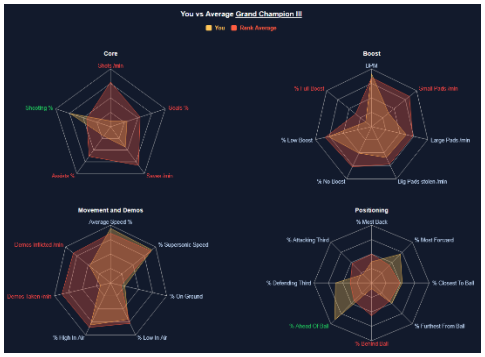


Six playstyles were derived from the interviews: a Striker prioritises scoring with high shots and goal ratio; an Enforcer creates openings through demolitions and bumps from the opponent's third; a Freestyler favours aerial plays, spending significantly more time high in the air than most; a Ball Chaser stays consistently closest to the ball; a Defender rotates back frequently, spending most time behind the ball and in their defensive third; a Playmaker creates chances through passes, reflected by a high assist count and time in the neutral third. Each playstyle includes unique general advice that may be difficult to conclude from statistics alone.

A Random Forest Classifier was chosen for its reliability, resistance to overfitting, and native support for feature importance and model confidence extraction, directly enabling the "Why this playstyle?" and "Other playstyles" parts of the section. All features are expressed as percentiles calculated within each rank, meaning the model inherently accounts for the fact that statistics vary significantly across ranks without requiring rank as an explicit feature. Training data consisting of over 400 players was constructed by averaging each player's statistics across their 10 most recent games in a given game mode, reflecting the same aggregation principle that underpins the project, and manually labelled using a custom-built labeller with full details of both in the implementation section.

Rank Comparison

The Rank Comparison section compares the user's averaged statistics against the rank average using spider charts across largely the same statistics as the Data Overview section. Contrasting colours and a clear legend differentiate the user from the rank average, with each chart titled to correspond to its category. The user can select any rank for comparison via a dropdown, with the spider charts updating automatically. Statistics falling within the top or bottom 20% of their rank are highlighted as outliers in red or green depending on direction, to bring attention to those areas, as those are most likely to need changing for improvement purposes.



Implementation

Upload Page

Header and Upload Requests: When a replay file is uploaded, the system sends two asynchronous requests to the backend simultaneously: a header request, which extracts basic details such as the player list, replay name, game mode, and date (falling back to the map name via map code if the replay name is empty), and a full parse request. Since the header request is near-instant, each replay appears in the list immediately while parsing continues in the background, keeping the interface responsive and allowing the user to continue uploading. Although the header and full parse requests

are asynchronous to each other, full parse requests are processed synchronously among themselves — experimentation showed no difference in total parsing time, and this approach allows the loading bar to accurately reflect progress based on the number of replays parsed out of the total.

Caching: Parsed replay data is cached in the user's IndexedDB using the unique game ID as the key, so repeated uploads skip parsing entirely and retrieve data directly from storage. This is intentionally only applied to file uploads, as the Ballchasing.com upload method retrieves already-parsed data via the API, making caching unnecessary.

Ballchasing Upload: The Ballchasing.com upload method was added late in implementation, primarily to address unexpectedly slow parser performance on the hosted server, and additionally to improve consistency with the large pre-computed dataset, which was also built using the Ballchasing.com API and therefore the same parser. It is also worth noting that many players have replays on Ballchasing.com without their knowledge, as others can upload games they were present in — as an example, I personally have over 2000 replays on Ballchasing.com without having uploaded a single one, making this upload method more broadly useful than it may initially seem. Another important aspect of this feature is the way it was integrated late into the implementation stage; all the field names were mapped from the Ballchasing.com API names to the Carball parser names using a large dictionary in the backend held in “format_ballchasing.py”, so that the rest of the code required minimal changes to accommodate replays uploaded in this way.

Player Dropdown: The player dropdown is populated using the union of all player IDs across uploaded replays rather than the intersection, so that accidentally uploaded replays the user wasn't in don't result in an empty dropdown. The system simply ignores replays the chosen player isn't present in, which has minimal impact since individual replays are just additional samples in an aggregated analysis. The game mode is specified manually via a toggle rather than inferred automatically, since private matches can be recorded as 4v4 despite only having 3 players per team, making automatic inference unreliable.

Error Handling: Error handling is designed to ensure the analysis stage receives only valid data. Invalid file types and duplicate replays are caught before any request is made, displaying a short inline error message. Parser failures, API errors, and missing player or rank selections each display appropriate messages while keeping the system fully operational. Errors are automatically cleared when the user takes corrective action.

Data Overview

The Data Overview is driven by two constants: DATA_SKELETON, which initialises all chart data at zero in the exact structure expected by Recharts, and categoryCharts, which maps each chart to its title, chart type, and corresponding location in the skeleton, allowing the entire section to be rendered iteratively given the large number of charts. The replay data is iterated game by game, with the target player, teammates, and opponents identified per replay and used to populate a deep copy of the skeleton via functions in a separate dataPopulating.tsx file. These functions use a COLUMN_NAMES constant that maps each statistic to its chart location, display name, and raw data field name, keeping the aggregation logic consistent. Each value is divided by the number of replays as it is added, and teammate and opponent values are additionally divided by their respective counts, meaning the skeleton holds fully averaged values once population is complete. Proportional statistics are expressed as percentages and displayed using stacked bar charts. Some normalisation is applied; average speed is converted to a percentage of the maximum possible speed rather than an arbitrary raw value, and total boost consumption is divided by game duration to give a per-minute

figure more familiar to players. The Possession category is conditionally hidden when Ballchasing-uploaded data is detected through a “ballchasing” field in the data, as Ballchasing does not provide per-player possession data.

Playstyle Classification

Data Collection: Training data was collected using a custom script querying the Ballchasing.com API, gathering data for players from Gold I upwards across 16 ranks, ranks below this were excluded as players at that level are still developing mechanical fundamentals and would benefit less from the project. For each rank, 30 players were sampled from recent replays, selected only if they had at least 10 relevant games listed, with the 10 most recent chosen and games of 5 minutes or under excluded to filter out forfeits and abnormal matches. Statistics were averaged across these games, maintaining the aggregation philosophy of the project, and immediately written to the CSV to avoid data loss in the event of the script failing. The resulting dataset serves as both training data for the classifier and the source of rank averages used in the Rank Comparison section, with this process run separately for 2v2 and 3v3.

Labelling: Before full labelling, a prototype model was trained on 50 manually labelled 3v3 players to verify there was sufficient signal in the chosen statistics, achieving approximately 65% cross-validation accuracy, enough to proceed confidently with labelling 400-500 players. A custom labelling tool was built for this, shown in Appendix E.1, which visualised each player's statistics as percentile bars coloured between red, grey, and green, making playstyle judgements far more intuitive than reading raw values. The labeller requested unlabelled data from the backend sequentially, advancing once a playstyle was assigned or the player was deleted. Labelling presented significant challenges, the subjective nature of playstyle classification led to inconsistent early decisions, resulting in multiple relabelling attempts. The ability to delete noisy data was added throughout the process, targeting players with completely average statistics or those suspected of playing below their actual rank, which ultimately produced a significant improvement in model accuracy.

Model Selection and Evaluation: Logistic Regression and Random Forest Classifier were trained and evaluated side by side for direct comparison, using an 80/20 train-test split, a custom top-2 accuracy metric that considered the second most likely playstyle if within 15% of the top prediction, and 10-fold stratified cross-validation to check for overfitting. Although both achieved similar accuracy, testing both in the live system revealed a clear practical difference - Logistic Regression consistently assigned 95–100% confidence to a single playstyle, even when statistics suggested a mix, whereas Random Forest naturally distributed confidence across multiple playstyles, which was far more informative. The Random Forest's feature importance reasoning was also noticeably more coherent, likely because Logistic Regression's linear nature prevented it from capturing the more complex relationships between statistics. Random Forest was therefore selected as the final model.

Initially, models were trained on raw statistics with rank as an explicit feature, since the same value carries very different meaning across ranks. However, the model overestimated its ability to account for this, frequently misclassifying high-ranked players with average within-rank aerial statistics as Freestylers. Switching to within-rank percentiles as features, calculated against each player's rank and its immediate neighbour, resolved this, as rank context became embedded in the data itself rather than requiring the model to learn it. This improved both accuracy and the coherence of the confidence and feature importance outputs.

Model Sureness and Feature Importance: Model confidence is derived naturally from the Random Forest's 300 trees as the proportion of trees voting for each playstyle gives a probability distribution

across all playstyles, producing the informative mix shown in the "Other playstyles" section. Feature importance is computed using a SHAP TreeExplainer, which measures each feature's contribution specifically to the user's prediction. Only features with a positive SHAP contribution are shown, as negative values indicate a feature pushing against the classification and would be misleading to present as a reason for it. Each feature's contribution is expressed as a percentage of the total positive SHAP mass rather than the raw value, so the user can understand the relative significance between features at a glance. High-direction features are prioritised as they directly support the classification, with any remaining slots from the top 5 filled by low-direction features if insufficient high-direction ones exist. The percentile value is displayed alongside each feature for context, with the underlying raw value visible in the Rank Comparison section.

Rank Comparison

The user's statistics are aggregated in the same way as in the Playstyle Classification section. Two backend requests are then made: one to retrieve rank average values and their percentiles, and one to convert the user's raw values into percentiles.

The rank average endpoint reads from the pre-computed dataset, filters to the specified rank, and computes the mean of each statistic across all players at that rank. Each mean is then expressed as a percentile across the entire dataset rather than within the rank alone - since the mean is generally close to the median, using a within-rank percentile would place almost every statistic near 50%, producing uninformative spider charts. Expressing it across all ranks instead scales the rank average proportionally, which is most apparent when switching the comparison rank, as the spider chart shape changes meaningfully.

The user percentile endpoint receives the user's raw values and computes two percentiles per statistic: one against the entire dataset, used to position the user's shape on the chart, and one against the user's own rank, used to colour the axis labels - green for the top 20% and red for the bottom 20%, highlighting outliers in each direction. When the user selects a different rank for comparison, only the rank average is re-fetched and combined with the already-computed user data, avoiding redundant recalculation. A tooltip on each axis displays the raw values for both the user and the rank average, so the actual figures remain accessible alongside the percentile-based chart.

Testing

Upload Page Functional Tests

Test	Expected Result	Pass/Fail	Evidence
Upload valid .replay file	Replay appears in list immediately with header details, data is parsed.	Pass	F.1
Invalid file type	Appropriate error message appears and no request sent.	Pass	F.2
Duplicate file	Confirm error message appears and no request sent.	Pass	F.3
Valid Ballchasing URL/ID	Replay name appears in list immediately with correct name, data is stored in replay data list.	Pass	F.4
Invalid Ballchasing URL/ID	Appropriate error message appears, no data added to replay data list.	Pass	F.5
Missing player/rank selection on submit	Appropriate error messages appear and data analysis does not begin.	Pass	F.6
Caching works	Upload same replay file twice, second should be retrieved correctly from IndexedDB	Pass	F.7
Deleting replays	Removed replay is not included in final replay data list and therefore not considered for analysis.	Pass	F.8

Parsing performance	Average parsing time per replay between 2-3 seconds. **Actual result:** 7 seconds average per replay.	**Fail**	F.9

The parsing performance test failed due to the slow performance of the replay parsing library. As a mitigation, Ballchasing uploads were used to obtain pre-parsed data, albeit with certain trade-offs. These are explored further in the Evaluation section.

Analysis Page Functional Tests

Test	Expected Result	Pass/Fail	Evidence
Data parsed and aggregated correctly	Use replays with known values and upload them, the Data Overview values should show correct averages.	Pass	F.10
Playstyle Classification returns appropriate result	Check if feature importance result is sensible given the playstyle and confirm with Rank Comparison section.	Pass	F.11
Choosing different ranks in Rank Comparison section	The spider charts update automatically and correctly.	Pass	F.12
Outlier colours appear correctly	Confirm values are sufficiently larger for outlier statistics.	Pass	F.13

Model Accuracy Testing and Evaluation

The trained models were evaluated using an 80/20 train-test split and 10-fold stratified cross-validation, reporting both mean and standard deviation. A custom Top-2 Accuracy metric was also introduced, counting predictions as correct if the second-highest playstyle was correct and within 15% confidence of the top prediction, accounting for players with mixed playstyles. Classification reports were generated to identify underperforming playstyles and inform dataset relabelling. Full results for both 2v2 and 3v3 datasets are provided in Appendix F.16. All accuracies and cross-validation means exceeded 80%, surpassing the 75% requirement, indicating good generalisation, as well as an overall successful endeavour in classifying aggregated statistics.

Parser Consistency Test

The two parsers, Carball for file uploads and the Ballchasing parser for URL uploads, handle replay duration differently. Carball tracks statistics only during active play, whereas the Ballchasing parser includes kickoff countdowns and goal celebrations, which pads certain statistics in ways that are difficult to account for precisely. A side-by-side comparison of the same replay processed through both methods is included in Appendix F.14. The resulting playstyle classification was identical across both, though with differing confidence values, indicating that despite the parser differences the system produces consistent classifications regardless of upload method.

Unit Tests

To ensure backend robustness, unit tests were written for each of the six endpoints using FastAPI's TestClient and pytest. Each endpoint has tests covering three areas: a successful response with valid inputs, basic sanity checks on expected functionality (such as computed percentiles or a known field matching an expected value), and appropriate error handling for invalid inputs, for example, returning 404 for unknown ranks or replay IDs, 422 for malformed request bodies, and 500 for files that failed to be parsed. The full test list is provided in Appendix F.15.

Evaluation

End-user Testing and Evaluations

Four players tested the system and provided feedback, with full responses in Appendix G.1. Reception was broadly positive, with particular praise for the radar charts enabling immediate

identification of statistical outliers, and the playstyle classification being seen as genuinely actionable rather than just presenting data. The accuracy of playstyle descriptions, feature importance justification, and playstyle-specific advice were highlighted as effectively bridging the gap between statistics and practical in-game decisions.

Suggestions for improvement included additional statistics beyond those available through the Ballchasing API, such as interceptions, challenges, bumps, and possession transitions; consideration of effective playstyle combinations against specific teams; heatmap-based positional analysis compared against high-performing players; and a profile system for saving and comparing datasets across sessions to track improvement over time. The first and third suggestions would meaningfully enhance the project's goal but would require a custom parser, discussed further in the Future Work section. The profile feature would be useful to include but would require authentication and database implementation.

Limitations

Parser Inconsistency

The two parsers produce broadly comparable results, as evidenced by identical playstyle classifications from the same replay regardless of upload method. However, the inconsistency remains a meaningful limitation: Carball tracks statistics only during active play, whereas the Ballchasing parser includes kickoff countdowns and goal celebrations, padding certain statistics in ways that are difficult to quantify. Since the Ballchasing parser is closed-source, attempts to adjust Carball's behaviour to match produced mixed results and were abandoned.

This has a direct implication for the training data, which was collected entirely via the Ballchasing API, meaning the model was trained on data from one parser and may receive data from another at inference time. While testing suggests this does not affect the final classification, it could subtly affect confidence values and feature importance outputs in ways that are difficult to measure without a ground truth.

Slow File Parser Performance

Testing showed an average parsing time of 6–7 seconds per replay on a mid-range computer, which exceeds the 2–3 second requirement. While this is partially attributable to the performance constraints of the free hosting tier, the parser itself is the primary bottleneck. At scale, this would become increasingly problematic if multiple users were parsing large batches simultaneously. Potential improvements include parallelising the parsing of multiple replays and optimising the parser implementation itself, though the latter is limited by the fact that Carball is a third-party library with little control over its internals.

Biased Labelling

Having over 800 players labelled by a single person introduces inherent bias, as classification decisions are influenced by one individual's interpretation of each playstyle. Distributing labelling across multiple people, each assigned an equal share of players per rank, would produce a more representative dataset, while varied opinions would introduce more noise individually, consistent labelling decisions across labellers would naturally emerge as the dominant signal, reducing the effect of any single person's bias and producing a more objective model overall. This would also allow a greater number of players to be labelled in less time, further reducing the influence of noise through a larger sample.

Future Work

Custom Parser

The most impactful future improvement would be the development of a custom replay parser. The current reliance on two third-party parsers introduces the inconsistency discussed in the limitations section, and performance constraints from Carball directly contributed to the slow parsing times observed during testing. A custom parser would resolve both issues simultaneously by providing full control over how and what data is extracted.

Beyond resolving existing limitations, a custom parser would unlock a significantly richer set of statistics. Both Carball and the Ballchasing API provide aggregated totals computed after the fact, whereas a custom parser operating on frame-by-frame replay data could derive more granular statistics, as well as unlocking more advanced analysis using specific in-game moments. The inclusion of both would meaningfully improve both the playstyle classification accuracy and the depth of analysis available to the user.

Unsupervised Machine Learning approach

Rather than classifying players into manually defined categories, an unsupervised approach could discover natural clusters within the data, producing playstyles that emerge organically and removing the subjectivity and labelling bias discussed in the limitations section. This could also reveal behavioural groupings that a human labeller would not have considered.

The primary challenge is that aggregated replay statistics may not provide sufficient separation for clustering algorithms to identify meaningful groups, as many players exhibit mixed tendencies throughout each game that blur boundaries between playstyles. Frame-level data from a custom parser, as discussed above, would likely make this more viable by exposing specific periods of games that better differentiate players at a finer level of detail.

Overall Conclusion

The project successfully delivers on its core goal of providing players with aggregated, interpretable feedback on their gameplay tendencies across multiple games. All major features were implemented and received positively, with the playstyle classification in particular proving more effective than initially anticipated. While limitations exist, notably around parser performance and labelling bias, the system is functional and genuinely useful in its current state, with clear and well-defined directions for future improvement, especially through the implementation of a custom parser.

Appendix

Appendix A – Interviews

A.1 – Interview questions

Question index:

1. Do you use ballchasing? Have you used it to improve? If so, how? If not, why?
2. Features that could be added/improved so you can use it to improve?
3. List of playstyles you can label most players with?
4. Which aspects of their gameplay would give a player a specific playstyle?
5. What is a typical flaw from each playstyle / something they may struggle with?
6. Stats that differ most between ranks?
7. General advice for any specific ranks?
8. Most useful statistics to compare to others to highlight flaws?
9. What would x statistic being different tell us about the player's possible habits/flaws?

A.2 – Interview 1, 2 responses and evaluation

Finn (Interview 1):

1. Yes, yes, to see and improve boost management,
2. possession metric, to see which team is clearly dominant on the ball,
3. passive, aggressive, mechanical
4. PASSIVE: composed to challenge, waits for their teammate to make plays and plays around them,
   AGGRESIVE: In the opponents halve the majority of the game, will usually play ball-side and demo from behind
   MECHANICAL: Will usually make a play in the air to get over the opponent
5. PASSIVE: Will usually get beat to the ball because theyre not quick enough, also may struggle offensively to get goals,
   AGGRESIVE: Might leave the goal open or leave their teammate in an awkward position MECHANICAL: If they fail their play then it could lead to make their teammate awkward
6. Lower ranks tend to use more boost while supersonic, collect less mid pads, and have less shots on target than higher ranks,
7. I would say lower ranks needs to play around their teammates, be patients, learn to hit the ball hard and 50/50 slowly to win the ball, not to get it up the field,
8. small pads collected
9. wasting boost, not know their way around the map (not rotating usefully)

Jake (Interview 2):

1. It certainly has! The website made me realise I wasn't actually using powerslide, and that discovery really improved my movement by quite a bit.
2. An AI personalised summary on how each player could improve
3. Enforcer, Wasp, false 9, Sweeper keeper, Decoy
4. Enforcer: Loves a good demo. A physical presence who thrives on chaos and disruption.
   Wasp: Aggressive and relentless. (Ball chaser)
   False 9: Drops deep to link play, creates chances, and somehow still ends up scoring most of the goals.
   Sweeper keeper: A keeper that gets bored just sitting in net.
   Decoy: A skilled player who misses the ball completely, yet manages to confuse everyone in the process.
5. Enforcer: Ignoring the ball and chasing the man.
   Wasp: Can lead to double commits and frustrated teammates.

False 9: Teammates often get sidetracked chasing MVP instead of focusing on the game. Sweeper Keeper: Messes up the rotation.
Decoy: Likely given to the worst player on the team

6. Time in the Air, Time supersonic (speed of play)
7. I'd advise bronzes to turn on ball cam (click triangle / Y on Xbox) and practice powerslides (click square / X on Xbox)
8. Boost pads picked up during games (large and small), Boost amount used while supersonic Time on floor & air
9. Time spent on the floor versus in the air can reveal how confident a player is with aerial manoeuvres.

Finn and Jake evaluation:

An overall data overview may be useful for all levels of players, considering Finn is the highest rank, while Jake is a middle rank. The sort of data included in the overview can be similar as ballchasing.com, such as boost metrics to help with boost management and other general statistics, things like the lack of powerslide use should be covered in the data comparison section of my project, where Jake's lack of powerslide would be highlighted.

The personal AI summary could be a fun feature to include at the end, by implementing an existing LLM to make one based on information from the replays written in a way it can make an accurate summary, though accuracy must be investigated, and the feature would be overall of low priority to include.

The "aggressive" type of playstyle was mentioned in both, and the suggestion that this playstyle would include a player that stays on the opposing half of the pitch is a good one and can be differentiated from an "enforcer" through their number of demolitions, as well as the distance from the player to the ball on average.

Since I will be using a model to predict the rank of the replay/set of replays, a good set of features would include the average speed of a player; use of boost while supersonic is good for differentiating between lower and higher ranks; mid/small pads collected are a good indicator of the level of boost management of a player, which can help differentiate the rank. Time spent in the air is also useful since the higher the level of player, the more typical it is for them to be comfortable in the air, although there are exceptions in every rank so should be used carefully.

Small pads are very useful to highlight in the data comparison for same reason they can be used to differentiate rank well.

A.3 – Interview 3 response and evaluation

Adam (Interview 3):

1. yes, although not as much with the 10 replay limit they added. I have tried to although only once or twice. Not sure if it counts but getting replays from when you were at your peak and watching them back in rl is helpful. In terms of on the website it was mostly looking at boost usage, positioning, time in air, average speed, basically all the stats that it summarises and comparing them to everyone else on the pitch. To see if there are trends in my gameplay, e.g super grounded (which i used to be when i used it a 1/2 years back) to get an idea of what i'm doing differently. Other than that i don't think theres anything else you can really use ballchasing for
2. one thing that mostly i'm interested to see is there's a johhnyboi video a while back about % behind ball and basically he believed in 3s that had a significant correlation to winning. Would love to see that summarised to see if there is an actual correlation between the 2. More broadly, analysis of multiple games at once or even your whole library of games would be amazing to improve as you can 100% see your playstyle over many games and days which means you can be confident in the things you need to improve. This would be difficult i imagine but comparison to an average player at that rank/mmr in all the stats would be super helpful, like in a spider diagram so you can see ur speed is bad but your time behind ball is good, etc.,
3. a) redicter/striker= someone always ahead of ball. hard to differentiate with bump heavy playstyles though.

b) Bumper/demoer= ahead of ball + demos/bumps.
c) aerial player= time in top 1/3 of the air.
d) grounded/dribbler=time on ground.
e) passer=low no. ball touches, touches with high speed.
f) ballchaser= closeness to ball.
g) big rotations= distance from ball.
h) low boost player (probably same as ballchaser)= boost .
i) high boost player (probably same as big rotations)= boost

4. ^
5. a) never back to defend, overcommits easily for hard shots,
   b) overcommits easily, all or nothing plays, usually not back to defend
   c) not back to defend, very postivie playstle, e.g not worried about defense
   d) doesn't force very often, hard to beat players especially multiple when grounded. limited aerial options when forced
   e) need teamates on some wavelength, often always looks for it even when not on. all or nothing play, 2 commit for 1 defender. won't shoot
   f/h) lack of speed, one bad chall=out of play. cutting teamate even when they have better angle,
   g/i) don't force enough, bad at pressuring ball, always gives space,
6. average speed, avg boost
7. hit ball hard. think about teamates boost, positioning, etc. watch opponents not ball= all ranks below ssl. ssl= as before+ everything else in the game
8. could be= % behind ball, would have to verify this though, look at johhnyboi vid for more info. idk would have to separate out things to see more, e.g recovery speed, (like time from 0 to max speed)
9. behind ball i think is because it means you're not out of play, always good defence and then if behind ball when attacking= in control of ball. also could show where avg ball position is, if in someone else's half probably likely they're on defence most of game=lose

Adam Evaluation:

The answer to question 1 shows a typical high level player using statistics to create their own conclusions, a lot of the suggestions in this question can be addressed in the data comparison section, as well as the data overview. The data overview will include the comparison between players in the replay, since this can still be useful to many players, as seen from the previous answers.

Adam also mentions that using replays from when you played very well (played at your peak) is useful to improving, so you can see why you were doing so well before in comparison to now. This could be included as a feature depending on the difficulty of implementation, where you can compare 2 sets of replays with each other, though it won't be of high priority and may be something to be considered beyond the project.

A high replay upload limit or the ability to enter your username with a date range so the site automatically gathers the replays from the ballchasing.com database would be a good requirement. The high replay upload limit should be easily applicable and useful considering it will be able to identify trends more easily and accurately, whilst the automatic implementation would highly depend on the ballchasing.com API.

Adam's suggestion in question 2 includes the comparison to other ranks, this is exactly what the data comparison section would achieve, and I think visualising the main statistics using spider diagrams is a very good idea, and I will try to make sure to include it.

A lot of the playstyle suggestions are useful, especially a-f, they line up with general ones I thought of as well as previous answers to this question, since many of these terms and labels are used frequently when describing a specific player. The characteristics related to each ones I generally agree with and will likely be included somewhat when creating the model that gives the user a playstyle.

I think the answer to q4 makes me rethink the unsupervised approach, since it shows how well you can narrow down the features needed in order to be able to label a player with a playstyle that they will be familiar with.

Also, the multiple pieces of advice included in q5 for each playstyle show that when giving general advice for one, I can also back it up with evidence, so specific advice can be highlighted more if there are specific statistics that can be shown to prove it. i.e: a player may be a bumper (enforcer), so if labelled so, and their time spent near their goal is much lower than average for their rank before a goal occurs, the site could give the advice that they aren't back to defend enough and provide that statistic in comparison to the average one.

Recovery speed is an interesting statistic that I think could be useful to investigate as a potential one to highlight. % behind ball could be interesting to investigate, since this statistic has a correlation to a team winning, however I think that the % time spent behind the ball could be caused by the fact that the winning team ends up having more pressure, which in turn causes this statistic to rise, so although this could be useful, it would take a lot of effort to determine the usefulness of it, especially if trying to give advice individual to a player.

Based on the suggestions given, I think that the data comparison and rank prediction sections can be merged together, taking away the feature of a model predicting a rank, and instead putting more of the project's attention on creating a useful comparison between the player and other players around the same rank, which would require that the player provides the rank for the replay beforehand, which would also be useful in the playstyle section, since comparing data of vastly different ranks could skew the result.

A.4 – Interview 4 response and evaluation

Connor (Interview 4):

1. Yes ive used it to improve my rotation through heat maps e.g. not staying one place at once or rotating into goal and mostly to look at more indepth stats of the games
2. An overall rating of how you played in each game
3. chasey, controlled, fast, grounded, aerialed
4. shadow defending a lot
5. hesitation or bad decision making
6. centre balls are more seen in higher ranks as a stat, long shots/clears are more seen in lower ranks because of less control
7. champ - focus on being consistent rather than flashy
8. probably percentage positioned in certain areas and percentage of time infront/behind the ball
9. that they either are too far up the pitch or are too close to the play/far away

Connor Evaluation:

The heatmaps would be useful to include in the data overview section, since they would highlight positioning patterns, however it would be hard to generalise this over many replays, and the implementation difficulty would be quite high. This is a feature included in ballchasing.com, so to improve it, I would need to find a way to display this over many replays, however this will lead to issues that will take away from the usefulness, which to address such that the visualisation and analysis is useful, would be very difficult and could be a project of its own, so I will consider but not prioritise this.

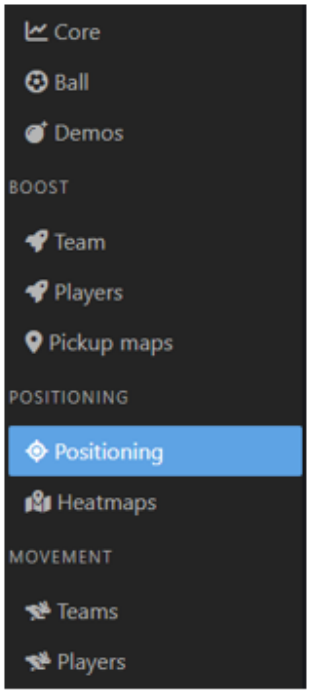
The advice for Champ level players will be useful, especially if I evidence this with data that shows they spend a lot of their time in the air and potentially have a low shot to goal ratio in the data comparison section, since this would indicate they may be focusing on the wrong part of their playstyle.

The rest of the answers solidify what was previously said or add to it, so will be considered overall.

Appendix B – Previous Solutions

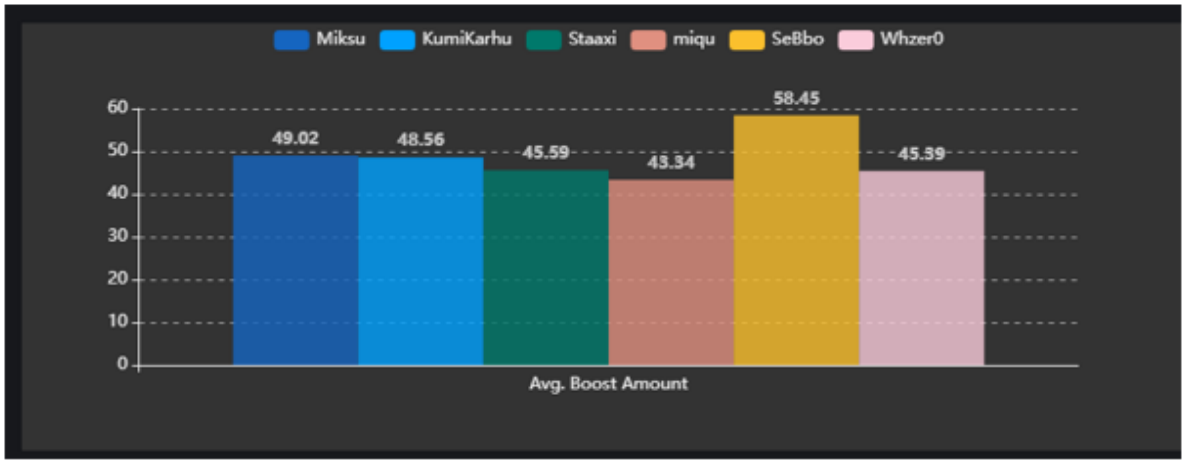
B.1 – Ballchasing.com

1.1 Filtering data by category



This is a useful way to prevent the user from becoming overwhelmed by the numerous statistics that the website provides.

1.2 Using Bar charts to visualise data

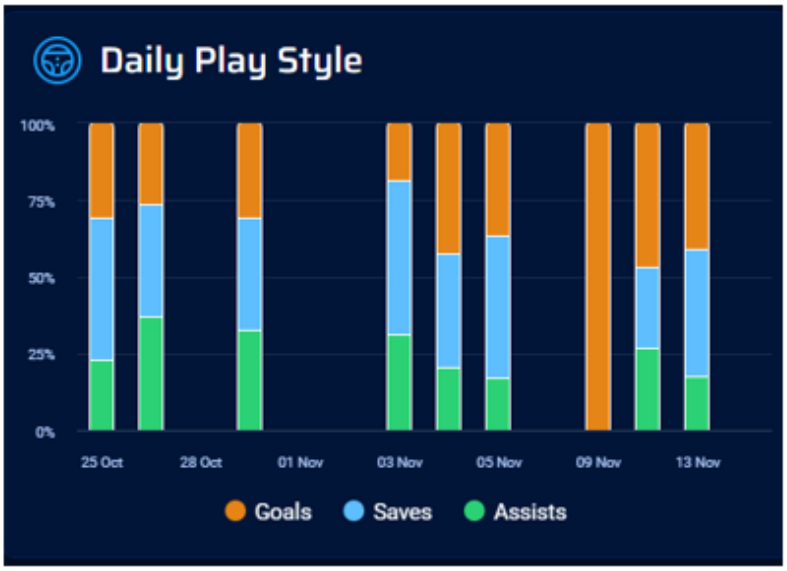


Almost all of the data is visualised and compared via the use of bar charts, this type of chart provides simple and effective comparison between the players.

The clear legend and contrasting colours help separate the data by the players, making comparison even more intuitive.

B.2 – Rocket League Tracker

2.1 Stacked Bar Chart (Goal to Assist to Save ratio) (Play Style)

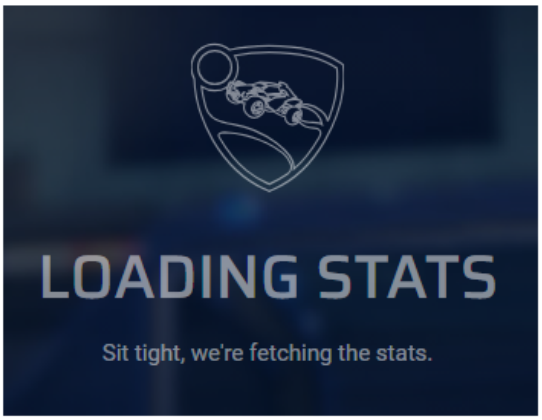


This website names the ratio between goals, saves, and assists the user's playstyle, which although can be partially indicative of how a user plays, this varies far too much game by game, and even viewing the overall pie chart does not tell you much about the user's actual play style.

However, a useful takeaway from this would be the use of stacked bar charts to display ratios across different days, which in my case may be useful to compare ratios of certain statistics between games lost and games won.

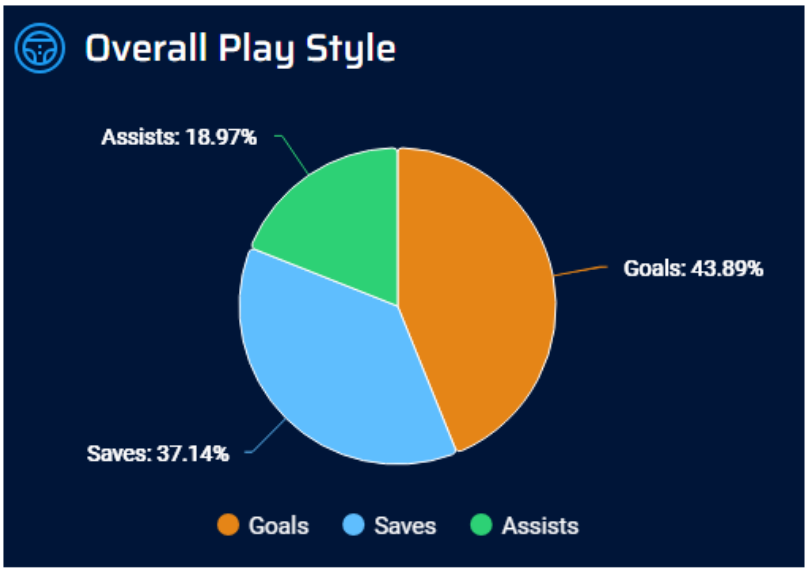
The contrasting colours are also useful to easily see the separation in the data, which is consistent across the other charts provided on this website.

2.2 Loading indicator



Since the website is required to fetch many statistics and visualise them, this can take a few seconds, and to make it clear the website is working, it uses this loading indicator to ensure the user that progress is being made. This will be useful in my project since I plan to allow a high maximum replay limit, which could take time to parse through and generate the charts for, so it would be useful for the sake of clarity to the user to provide an indication that progress is being made; if possible, I could extend this by somehow showing the user which replay the site is currently parsing, so the user has somewhat of an understand how long they will need to wait.

2.3 Flaws in this website's representation of playstyle



Like mentioned in 2.1, the ratio between these 3 simple and shallow statistics does not provide much detail as to how a user actually plays, and what they could do to improve, especially since most users will have very similar charts to this.

Additionally, the use of all data across the entire user's career does not provide an accurate representation of the user's current playstyle, since a player will always end up playing differently as they improve. This means that a user's playstyle from when they had just started the game is affecting what the website is showing them, making it almost useless to learn anything useful from at their current level, given they've improved/changed.

Another flaw to using data from so long ago is that players in general have changed and improved in each rank over time, this means that an average Diamond ranked player from 5 years ago is much different to the average Diamond ranked player today, which although not relevant to the pie chart, will be relevant when using training data in my project and should be considered.

To improve this, I will use a much wider set of statistics to indicate playstyle, as well as make it clearer what those statistics show about them. I will also use RECENT training data to ensure that the model is not skewed by outdated data.

Appendix C – List of Requirements

Project Specification

I = Interview **PS = Previous Solution**

1 Functional Requirements

1.1 Uploading Replays

1.1.1 Allow the user to upload multiple Rocket League replays **I3**
   a. Allow the user to upload 1 replay at a time
   b. Allow the user to upload a folder of replays

1.1.2 Only allow Rocket League .replay files to be uploaded

1.1.3 Only allow 1 type of game mode across the replays before analysing (2v2, 3v3)

1.1.4 Show names of the replays that the user uploaded so they can check they are correct **PS3.1**

1.1.5 Display appropriate error message for invalid upload attempt

1.1.6 Parse replay files using an appropriate library

1.1.7 (optional) Automatically gather replays from Ballchasing.com using its API given user provides their username and date range **I3**

1.1.8 Require user to choose their rank to allow rank-based comparisons and accurate playstyle classification

1.1.9 Guide the user to the location of their replays **PS3.1**

1.2 Data Overview (feature derived from I1 I2)

1.2.1 Visualise core gameplay statistics such as:

a. Average boost amount **I1**
b. Small boost pads collected **I1**
c. Boost wasted (used while going max speed) **I1**
d. Time spent on different boost ranges **I1**
e. Average speed
f. Time spent on ground/low in air/high in air
g. Time spent behind the ball
h. (optional) Possession (comparing user's team to opposing team) **I1**

1.2.2 Visualise the AVERAGE of the statistics across the replays using suitable charts **PS1.2**

1.2.3 Compare the user's average to their teammate's average and their opponent's average

1.2.4 Provide a way to compare certain statistics with games that were won and lost **PS2.1**

1.2.5 (optional) Allow users to upload 2 batches of replays to compare directly to each other **I3**

1.2.6 Allow the user to filter which data category is displayed **PS1.1**

1.3 Playstyle Classification

1.3.1 Use a supervised model to classify the user's playstyle, examples:

a. Striker **I3**
b. Enforcer **I2 I3**
c. Freestyler **I1 I2 I3**
d. Dribbler **I3 I4**
e. Ball chaser **I2 I3 I4**
f. Passive player **I1**
g. Playmaker **I3**

1.3.2 Use a narrowed down set of statistics to train the model, such as

a. Demolition/bump amount **I2 I3**
b. Time spent ahead of the ball **I1 I3**
c. Time spent high in the air **I1 I3**
d. Shot to goal ratio
e. Time spent low to the ground **I3**
f. Assists to total goals ratio
g. Distance to ball **I3**

1.3.3 Use manually labelled training data to train the model **I1 I2 I3 I4**

1.3.4 Only use the training data similar to user's rank

1.3.5 Display playstyle to user with explanation evidenced with relevant statistics

1.3.6 Provide general advice for improvement based on identified playstyle using suggestions from interviews

1.3.7 Use model sureness to show similarity to other playstyles

1.4 Player Data Comparison

1.4.1 Compare user's average statistics to average values similar to user's rank **I3**

1.4.2 Compare user's average statistics to average values of rank of user's choosing **I3**

1.4.3 Highlight the largest differences between user and average values in similar rank, to draw attention to potential areas of improvement
1.4.4 Visualise this with spider charts **I3**
1.4.5 Use data from large dataset with parsed replays to avoid using Ballchasing API each time
1.4.6 Provide advice based on the largest differences between user and average values in similar rank

1.5 Frontend + Backend Website

1.5.1 Frontend has an intuitive and responsive user interface
1.5.2 Backend built with framework suitable to handle machine learning, data analysis, and API calls
1.5.3 Frontend has suitable chart library

2 Non-functional Requirements

2.1 User Interface

2.1.1 Data overview isn't overwhelming – Allow user to pick which category of data to be displayed at any one time, not all at once **PS1.1**
2.1.2 Only display one section at a time (Replay upload, Data Overview, Playstyle Classification, Data Comparison)
2.1.3 Clearly distinguish between each section
2.1.4 Colours of text chosen such that it contrasts with its background
2.1.5 Colours in charts consistent **PS1.2 PS2.1**
2.1.6 Colours in charts contrasting for intuitiveness and accessibility **PS2.1**
2.1.7 Charts labelled clearly **PS1.2 PS2.1**
2.1.8 Adequate spacing for clarity
2.1.9 Each spider chart will include relevant types of data (boost, height in the air, speed) and cover all significant statistics
2.1.10 Any outliers will be highlighted using contrasting colour or other clear indicator

2.2 Performance

2.2.1 Support large batch of replays (20 or more) without excessive loading time (average below 2-3 seconds per replay uploaded) **I3**
2.2.2 Provide loading visualisation (optional: with progress) **PS2.2**
2.2.3 When comparing data, use data from the large dataset instead of using Ballchasing API for quicker loading speed
2.2.4 (optional) Cache charts for faster re-loading times
2.2.5 Only load section when first expanded to avoid longer initial waiting time

2.3 Accuracy

2.3.1 Train the playstyle classifying model with features that have largest impact, for both efficiency and accuracy
2.3.2 Playstyle model will be tested with testing data, requiring sufficiently high accuracy (at least 70% for example, due to subjective nature of the classification)
2.3.3 Use recent data to make the large, parsed dataset to ensure comparison is accurate to today's standards of different ranks **PS2.3**

2.4 Reliability

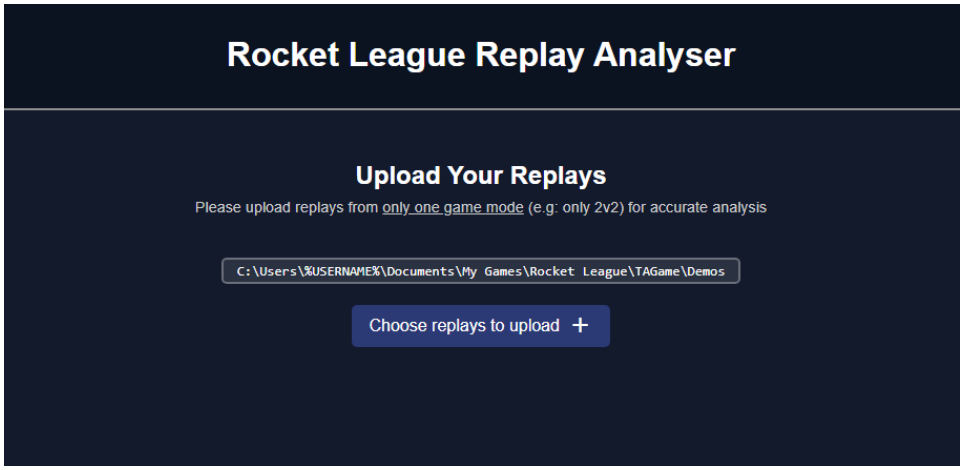
2.4.1 Handle API errors such that website remains responsive, and inform the user of any issues

2.5 Security

2.5.1 Training data must be anonymous
2.5.2 Large dataset data must be anonymous
2.5.3 User data should not be stored, only processed

Appendix D – Design

D.1 – Interface before uploading replays

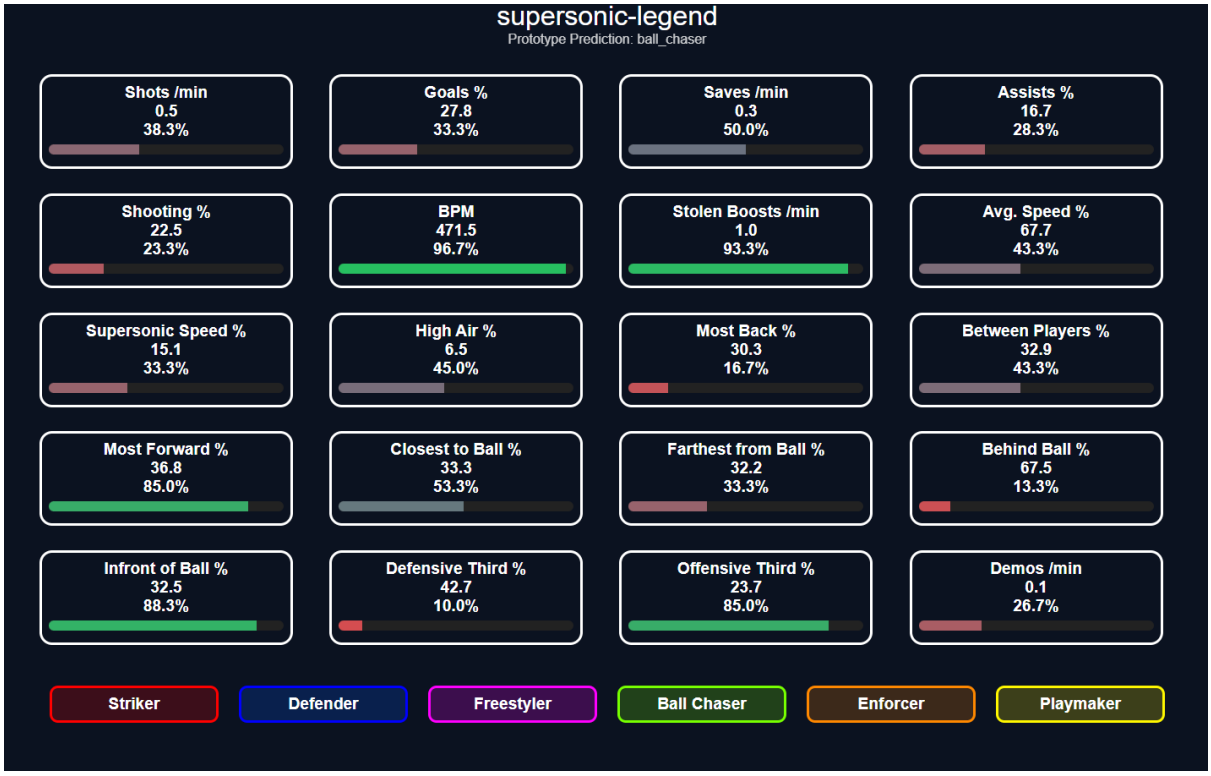


D.2 – Data Overview Category Data

Core	Shots, Goals, Assists, Saves
Boost	Boost used per minute, Small Boost Pads collected, Large Boost Pads collected, Time spent at 0 boost, Time spent at Low boost, Time spent at Full boost
Movement	Average speed (% of max), Time spent supersonic speed, Time spent at different heights (stacked bar chart)
Positioning	Position relative to teammates (pie chart), Distance from ball (pie chart), Behind/Ahead of ball (stacked bar chart), General Position on the field (stacked bar chart)
Demos	Demos inflicted, Demos taken, Large boost pads collected
Possession	Team possession time (pie chart), Average per player possession time, Total per player possession time

Appendix E – Implementation

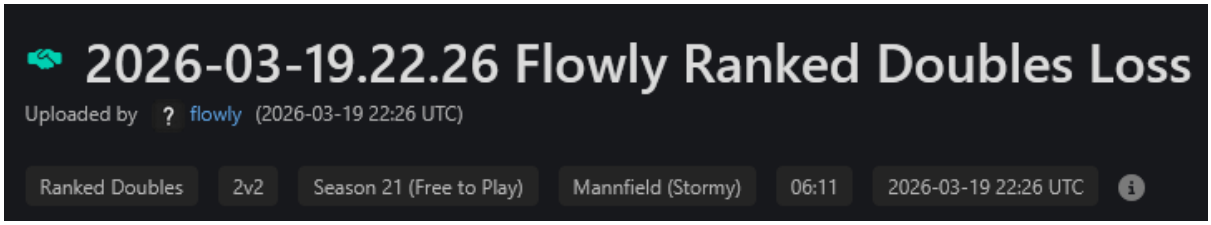
Appendix E.1 - Labeller



Appendix F – Testing

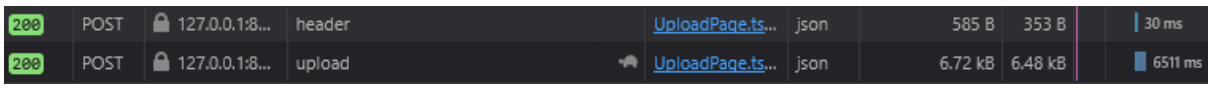
Appendix F.1 – Valid replay file

Replay name, gamemode and date:

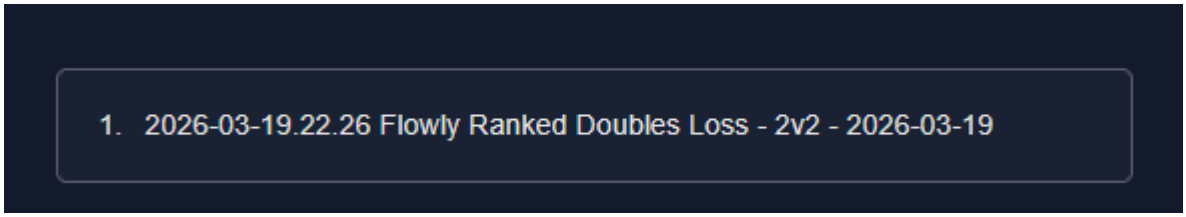


Downloaded replay file:

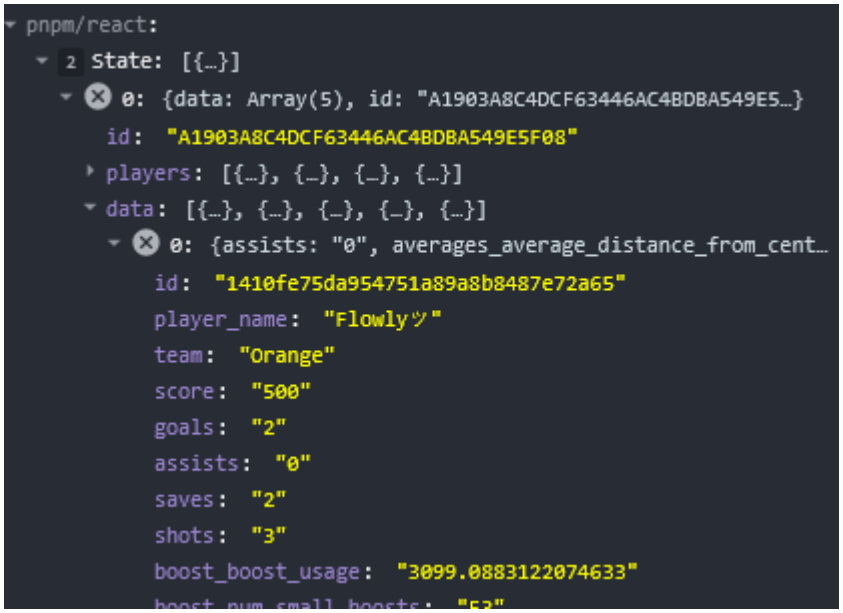
Requests made:



Correct details displayed on page:

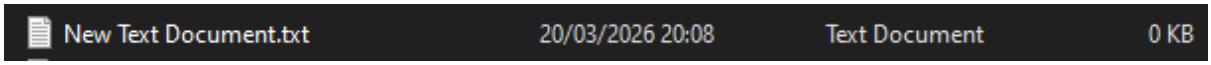


Data parsed:

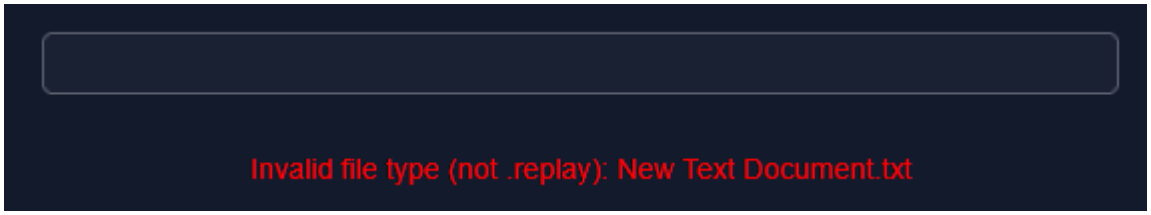


Appendix F.2 – Invalid file type

Invalid file:

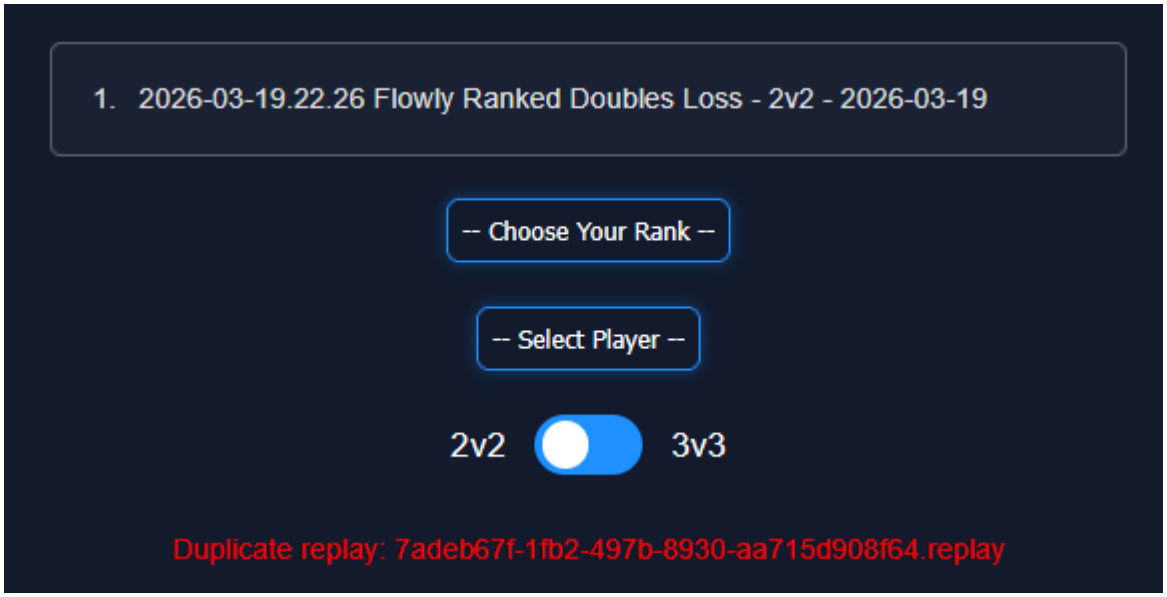


Appropriate error message:

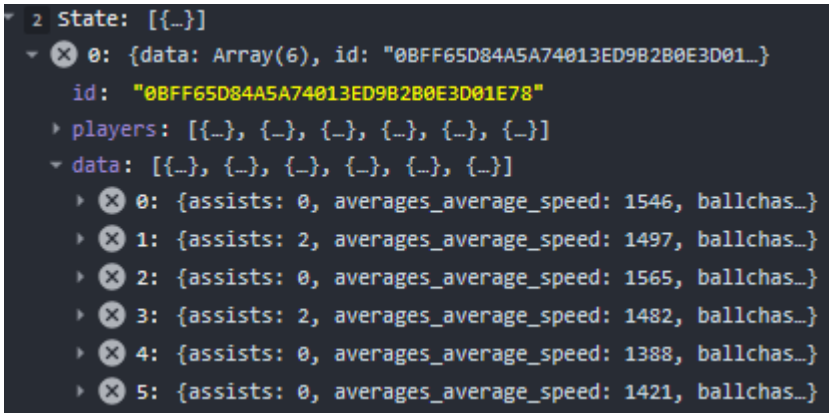


Appendix F.3 – Duplicate replay

Only 1 replay stays:

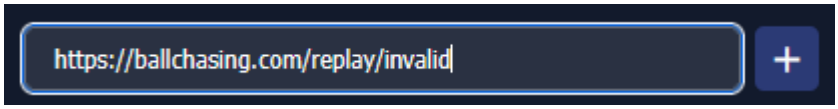


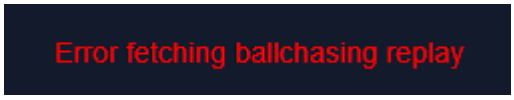
Appendix F.4 – Valid Ballchasing URL/ID



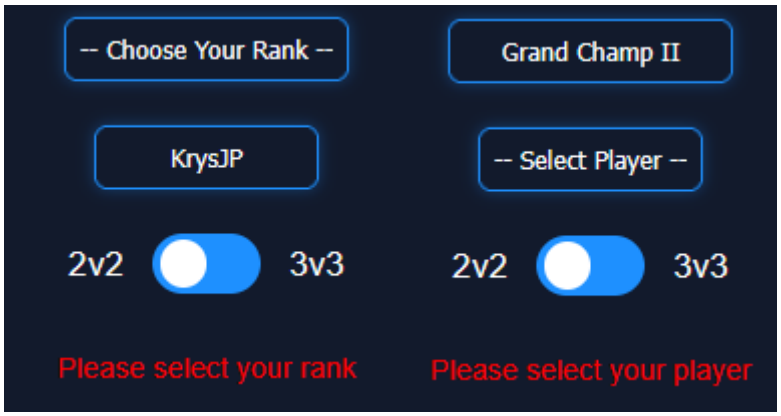
(note that Game ID != Ballchasing ID)

Appendix F.5 – Invalid Ballchasing URL/ID

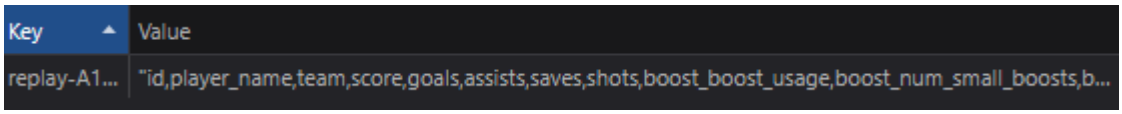


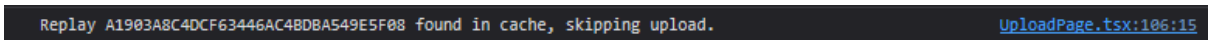


Appendix F.6 – No replays or missing player/rank selection on submit

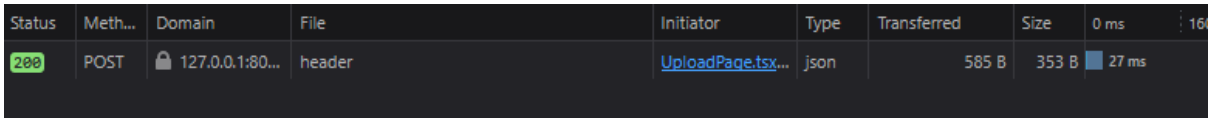


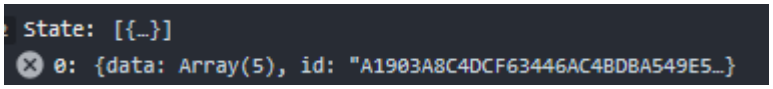
Appendix F.7 – Caching works



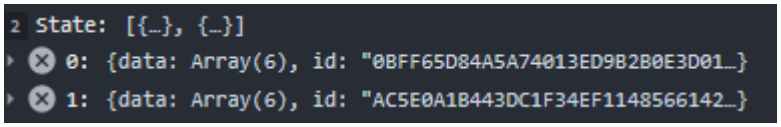


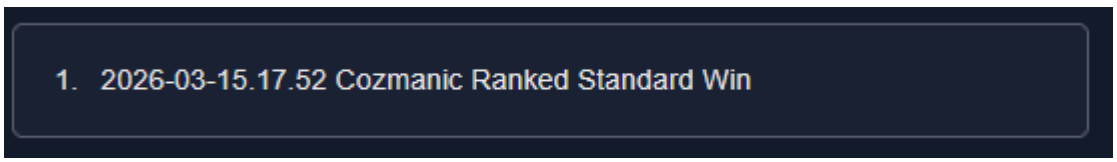
Only header request made (no parsing):



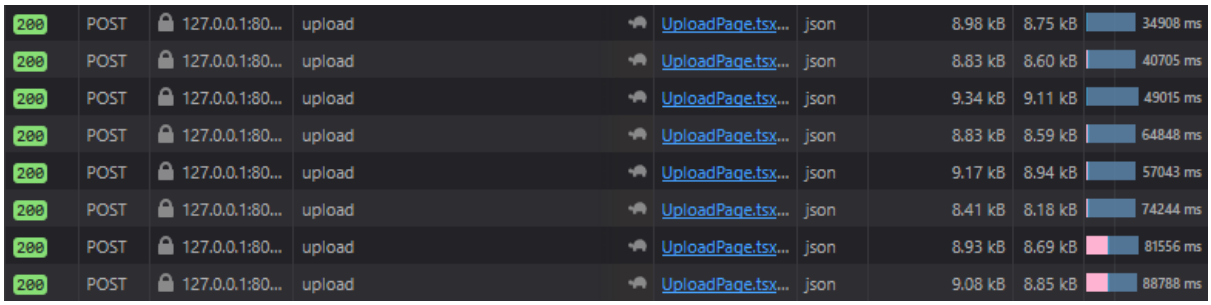


Appendix F.8 – Deleting replays





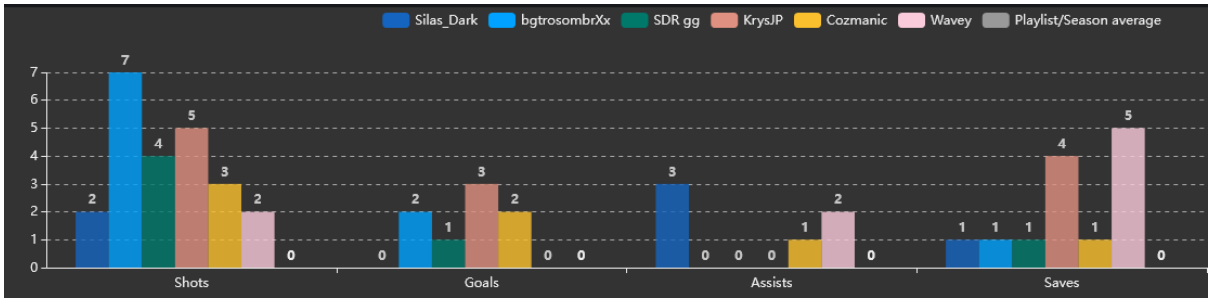
Appendix F.9 – Parsing performance

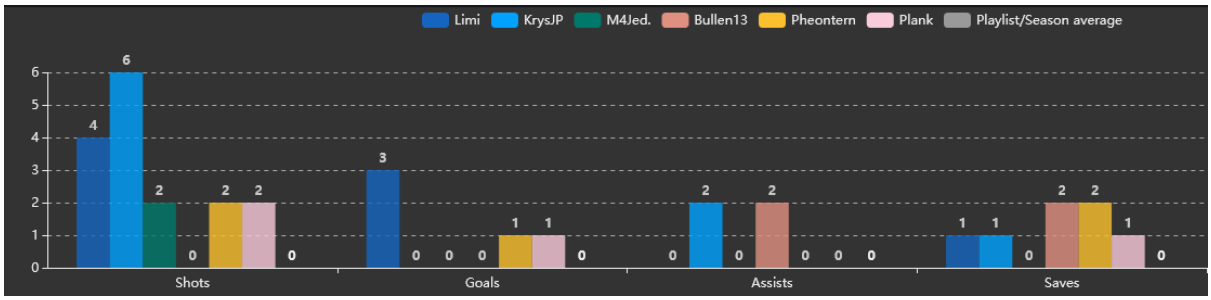


After the first request (34908 ms), each request afterward happened sequentially, with around 7000ms between them on average (7 seconds).

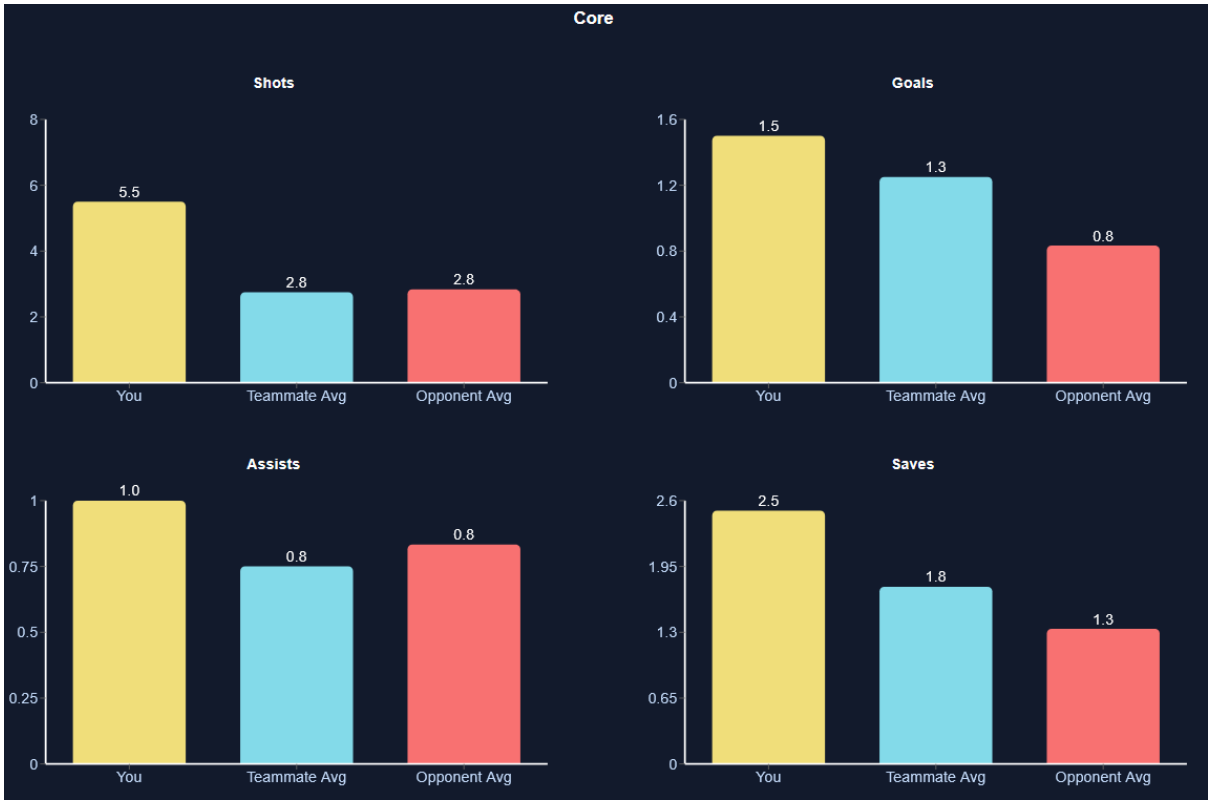
Appendix F.10 – Data parsed and aggregated correctly

Individual statistics sourced from Ballchasing:





Data Overview (with KrysJP as target player)



Total shots from User: 5+6=11 | Replays: 2 | Result: 11/2 = 5.5 | Correct

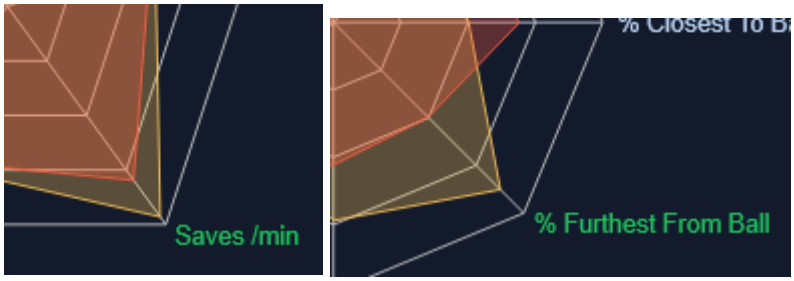
Total shots from Teammates: 3+2+2+4=11 | Replays: 2 | Teammates: 2 | 11/2/2 = 2.75 | Correct

Total shots from Opponents: 7+4+2+2+2=17 | Replays: 2 | Opponents: 3 | 17/2/3 = 2.83 | Correct

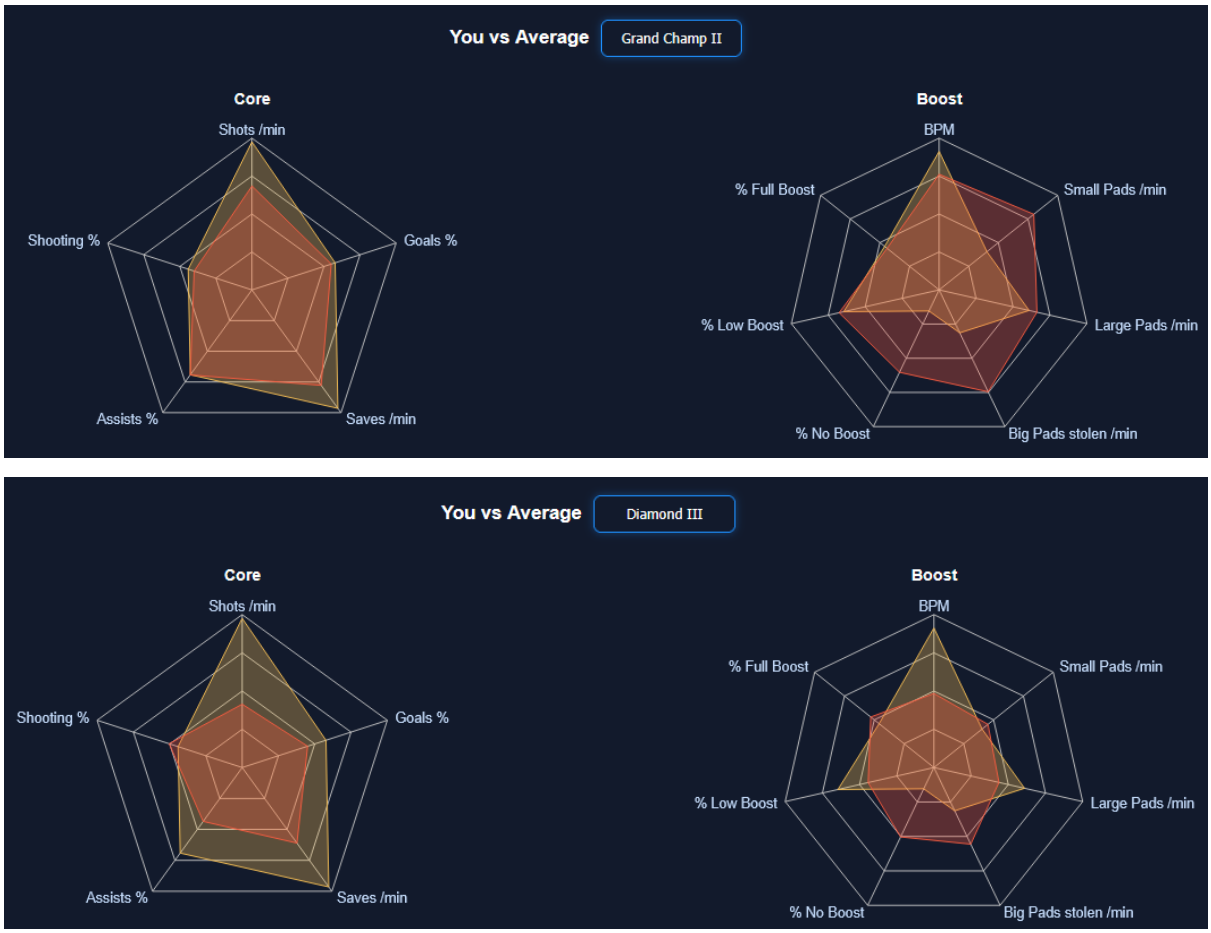
Appendix F.11 – Playstyle Classification return appropriate result

Defender for Saves /min + Farthest From Ball | Correct

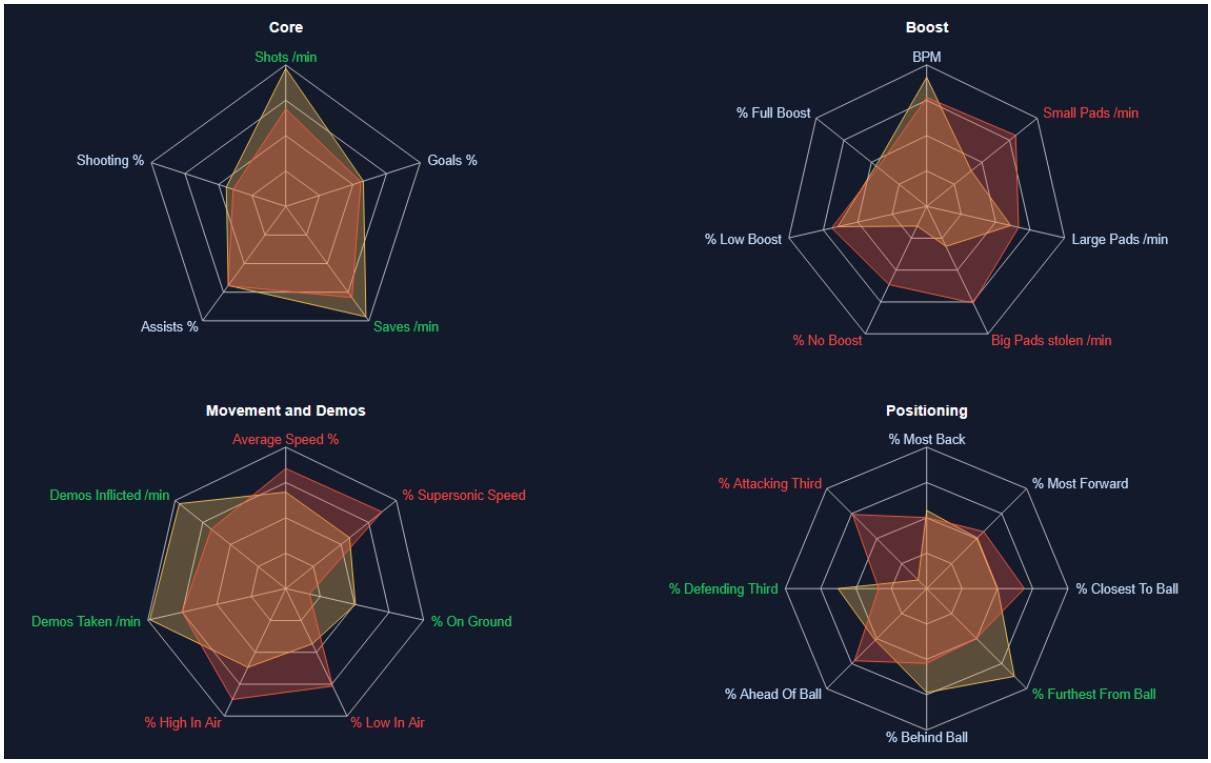
Rank Comparison confirms data correct and in top 20%.



Appendix F.12 – Choosing different ranks in Rank Comparison section



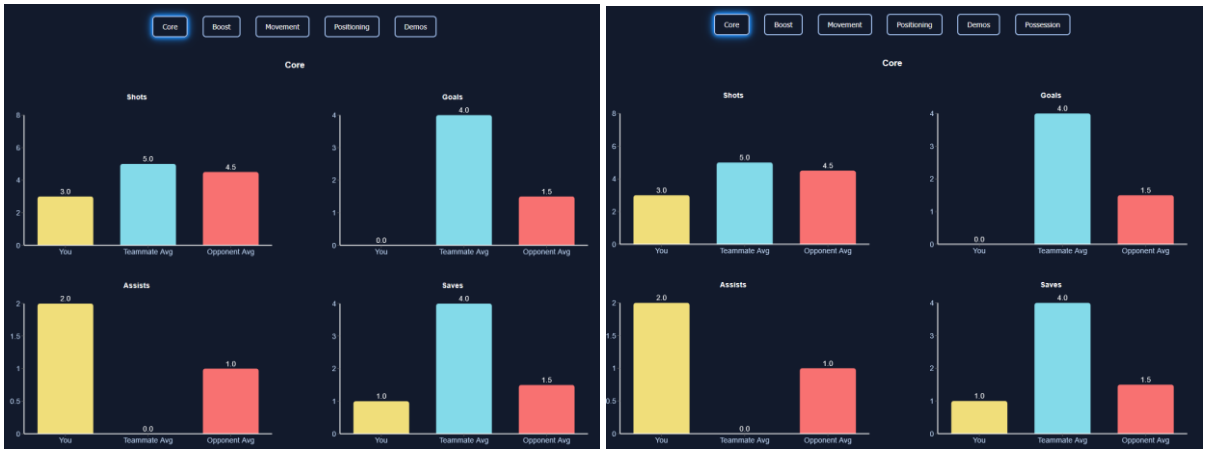
Appendix F.13 – Outlier colours appear correctly



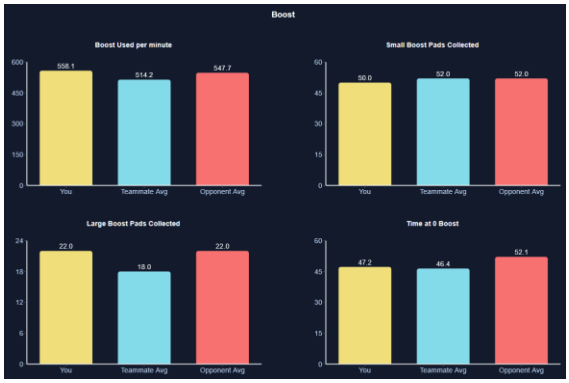
Clearly high/low values are highlighted as intended.

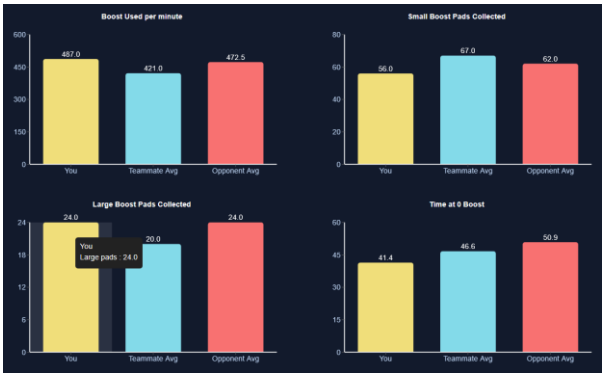
Appendix F.14 – Parser difference

Same values for core:

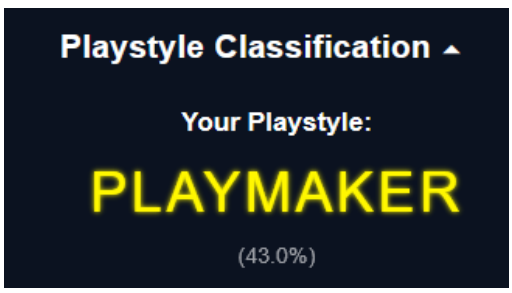


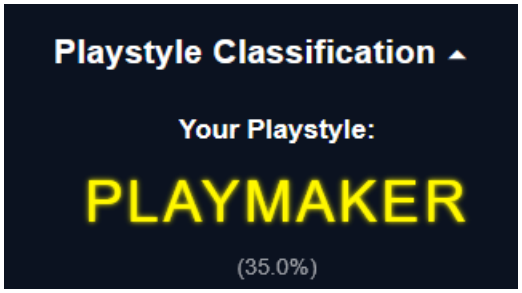
Largest difference is boost category:





Same playstyle result, but different confidence:





Appendix F.15 – Units Tests

```
main_unit_tests.py::TestUpload::test_success PASSED
main_unit_tests.py::TestUpload::test_correct_data PASSED
main_unit_tests.py::TestUpload::test_error PASSED
main_unit_tests.py::TestHeader::test_valid_with_name PASSED
main_unit_tests.py::TestHeader::test_valid_no_name PASSED
main_unit_tests.py::TestHeader::test_invalid_file PASSED
main_unit_tests.py::TestRankAverage::test_success PASSED
main_unit_tests.py::TestRankAverage::test_averages_and_percentiles PASSED
main_unit_tests.py::TestRankAverage::test_unknown_rank PASSED
main_unit_tests.py::TestUserPercentiles::test_success PASSED
main_unit_tests.py::TestUserPercentiles::test_correct_values PASSED
main_unit_tests.py::TestUserPercentiles::test_bad_radar_data PASSED
main_unit_tests.py::TestUserPercentiles::test_unknown_rank PASSED
main_unit_tests.py::TestPlaystyle::test_ordered_classes_probs_sum_to_1 PASSED
main_unit_tests.py::TestPlaystyle::test_ordered_classes_probs_descending_order PASSED
main_unit_tests.py::TestPlaystyle::test_invalid_player_data PASSED
main_unit_tests.py::TestBallchasing::test_success PASSED
main_unit_tests.py::TestBallchasing::test_correct_data_returned PASSED
main_unit_tests.py::TestBallchasing::test_correct_data_returned PASSED
main_unit_tests.py::TestBallchasing::test_correct_data_returned PASSED
main_unit_tests.py::TestBallchasing::test_invalid_id PASSED
```

Appendix F.16 – Model Evaluations (2v2/3v3)

```
===== Random Forest Classifier =====
Test Accuracy: 0.8048780487804879
              precision    recall  f1-score   support

 ball_chaser       0.75      0.82      0.78        11
    defender       0.94      0.94      0.94        18
    enforcer       0.76      0.87      0.81        15
  freestyler       0.68      1.00      0.81        13
   playmaker       0.88      0.58      0.70        12
     striker       0.88      0.54      0.67        13

    accuracy                           0.80        82
   macro avg       0.82      0.79      0.79        82
weighted avg       0.82      0.80      0.80        82

Top-2 Accuracy (0.85 threshold): 0.8048780487804879
CV mean/std: 0.8364024390243902 0.06792149232438642
```

```
===== Random Forest Classifier =====
Test Accuracy: 0.8028169014084507
              precision    recall  f1-score   support

 ball_chaser       0.89      0.80      0.84        10
    defender       0.86      0.75      0.80        16
    enforcer       0.90      0.75      0.82        12
  freestyler       0.75      1.00      0.86         9
   playmaker       0.73      0.73      0.73        11
     striker       0.73      0.85      0.79        13

    accuracy                           0.80        71
   macro avg       0.81      0.81      0.81        71
weighted avg       0.81      0.80      0.80        71

Top-2 Accuracy (0.85 threshold): 0.8591549295774648
CV mean/std: 0.8002380952380952 0.04987392772953191
```

Appendix G – Evaluation

Appendix G.1 – End-user tests

User 1:

It has a very high level of trend spotting and ability to reason why the playstyle has been selected, aswell as giving ways of how this playstyle could negatively impact you or your teammates, by either pointing out how decision making could be improved with selection of using common plays e.g. shooting or general pitch positioning.

Possible improvement: i feel like if it was possible to input more parameters, to show a more direct influence of a playstyle apart from the norm of ballchasings parameters that it pulls e.g. interceptions, possesion lost and gained, challenges, clears, bumps taken/given (not demos). Implementing more of these could swing percentage numbers for playstyles and increase the idea of potential flaws given.

User 2:

The webapp is very informative, showing how my playstyle can be beneficial but also detrimental in high level lobbies. The consideration and tips on improvement are insightful with general consideration on other playstyles. Also the figures/graph are attractive, with options to compare your gameplay with other ranks.

Possible improvement: Maybe consider the playstyle of other players in the chosen replay, and what combinations/percentages of playstyles would work best within that specific game to have a better chance at winning (idk how youd do this but it would be kinda cool).

User 3:

Honestly bro the radar charts are class because you can literally just look at them and go "oh right I'm way worse at saves than everyone else at my rank" without having to sit there wondering why i get terrible teammates, and the playstyle thing is proper useful because instead of just giving you a load of numbers it straight up tells you you're a ballchaser and here's why that's causing you problems, so you actually know what to work on next session instead of just vibing and hoping you get better.

Possible improvement: I think it might be too complicated but adding like a heat map of your car and comparing that to pros or people that are doing good in your rank would be better but being a computer science student I know that's going to be a lot of work.

User 4:

review: providing interesting and accurate descriptions of different playstyles and possible flaws to work on.

Improvements: provide profile so you can compare different data sets and compare days etc